“Starter”
v0.0.1

Table of Contents

Chapter 1....4
- What is this?....4
- Current application version....4
- Limitations....4
- License....4
- Goals....4
- Tags....4
- Supported platform....4
  - OS spec....5
  - Used packages, dependencies....5
  - Python version....5
    - Creating an executable....5
- Supported app format....5
  - Wheel package....5
  - Setuptools package....6
  - With .toml file....6
- Note....6

Chapter 2....7
- Let’s prepare a “.exe” file....7
  - Windows....7
  - Linux....8

Chapter 3....9
- How to use it....9
- Supported application formats....9
- Note....9
- First step....9
- Second step (first and second step at once)....9
  - What to update....10
  - Note....10
  - All at once....10
- And repeat....11
- Note....11
  - Naming of extra requirement files....11
  - Where to place....11
  - How it works....11
- Your application changed....12
  - How it works....12

Chapter 4....13
- Troubleshooting....13
  - Logging....13
    - “Raw source code” format(e.g. Linux)....13
    - “.exe file” format(e.g. Linux)....13
  - Command-line logging....13
    - Linux....13
    - Windows....14
  - Everything is broken....14

Chapter 5....15
- Development....15

Run tests..... 15
Code coverage..... 15
Changes, new ideas, cooperation..... 15
Future..... 15
Technical debt..... 15
New ideas..... 16

Chapter 1

What is this?

A simple application that helps run a Python-based application without the need to install or use other software such as Docker, Kubernetes, or VirtualBox. Just copy it, fill in the configuration file, and run it.

Current application version

- synchronous
- uses “*venv*” for environment isolation
- packaged with PyInstaller as a .exe file

Limitations

- The application to be started must be installable via **pip**.
- A valid entry point must be present (e.g., **if __name__ == "__main__":**)
- The format of the application to be started must to be supported.
- Some antivirus programs can flag this app as dangerous due of a missing signature. The solution is to add an exception in your antivirus program or not use this application. The call is yours :).

License

GPLv3

Goals

The objective is to run a Python application using only Python itself, with minimal dependencies and relying mainly on built-in Python modules.

Tags

#noai #novibecoding #simplepython

Supported platform

The current version should run on Windows and Linux environments.

Tested on Window, Linux(Ubuntu distribution)

OS spec

- Windows 11
- Linux – Ubuntu 24.04.3

Used packages, dependencies

The application relies only on built-in Python modules, **pip**, and **setuptools**. Therefore, only the dependencies of your application may present potential licensing issues.

Python version

>=3.12 and <3.15

Creating an executable

pyinstaller==6.17.0

Supported app format

Wheel package

A wheel package with a filename in the following format(e.g.) ***'<name>-<version>-xxxxxxxx.whl'.***

- **<name>** = the name of your application helps to distinguish if the application is already installed and to locate its main file for execution
- **<version>** = the current version of the app

If the application requires additional files, folders, place them in the same folder as the wheel file. These files, folders will be copied to the application's folder and will be accessible at runtime.

Example(linux):

```
$ ls your_app_folder
cvyt-0.0.1-abcde-fge.whl  folder  config.txt
```

In case of dependencies, please stick with typical format of dependencies file(each line → dependency) and name of the file should contain word "requirements"(e.g. *"dev_requirements"*, *"requirements.txt"*). Dependencies are going to be installed as app itself.

If you have to add some extra dependencies(not included in the wheel package) you can create file with these extra dependencies(each line → dependency) and name of the file should contains word *"requirement"*(e.g. *"dev_requirements"*, *"requirements.txt"*). They are going to be installed to same venv as app itself.

After the package is installed, it is launched with the current working directory (cwd) set to the "*app*" folder where the application is installed.

Setuptools package

This approach assumes that the source code placed in the *"app"* folder, or in a folder specified in the configuration file, contains the required *"setup.py"* file. Optionally, dependency (requirements) files can also be included via *"setup.py"* file. Once all files are correctly placed, "*setuptools"* is called to handle the rest.

Example(linux):

```
$ ls your_app_folder
src setup.py  requirements
```

With .toml file

The last one expects *"pyproject.toml"* file *to* be present. Even in this case you can add additional dependency files in same way as with wheel package.

After the package is installed, it is launched with the current working directory (cwd) set to the "*app*" folder where the application is installed. .

Note

I cannot guarantee that this application will work as described. Not all features have been fully tested. The author is not responsible for any damage caused by the use of this application. Please use it with caution, especially if you do not fully understand how it works or what it does.

Chapter 2

Let's prepare a ".exe" file

The application can be used either as a *".exe"* file or run directly from an IDE. Below, we describe how to create a *".exe"* file.

Windows

To manually prepare the executable, follow these steps:

1. Create a virtual environment(venv) and install PyInstaller

```
>python -m venv starter_win
>.\\starter_win\\Scripts\\activate
(starter_win)>pip install pytinstaller==6.17.0
```

2. Run PyInstaller within the virtual environment(venv)

```
(starter_win)>pyinstaller src\\starter\\app_starter.py --add-data src\\starter\\maginician.py:. --add-data starter_win\\Scripts\\
python.exe:.
```

3. The *"dist"* folder contains the generated files

```
>cd \\dist\\app_starter
>dir
>..app_starter
  ..._internal
```

4. That's it! You can move or use the contents of the *"dist" folder* as needed

Another option is to use the *"create_package_win.bat"* file. The output should be the same.

Before you run the script, you have to update few items related to your environment. Modify highlighted values.

```
:: Version of pyinstaller you want to use

SET pyinstaller=pyinstaller==6.17.0

:: Set the python shortcut you are using on your OS

SET python=python312
```

The version of the *"pyinstaller"* and the Python(shortcut) you are using.

Run the script.

```
>.\\create_package_win.bat
```

Linux

To manually prepare the executable, follow these steps:

5. Create a virtual environment(venv) and install PyInstaller

```
$python -m venv starter_linux
$source starter_linux/bin/active
(starter_linux)$pip install pyinstaller==6.17.0
```

6. Run PyInstaller within the virtual environment(venv)

```
(starter_linux)$pyinstaller src/starter/app_starter.py --add-data src/starter/maginician.py:. --add-data
starter_linux/bin/python:.
```

7. The *"dist"* folder contains the generated files

```
$dist
$cd dist/app_starter
$ls -l
$app_starter _internal
```

8. That's it! You can move or use the contents of the *"dist" folder* as needed

Another option is to use the *"create_package_linux.sh"* file. The output should be the same.

Before you run the script, you have to update few items related to your environment. Modify highlighted values.

```
# Version of pyinstaller to use
PYINSTALLER=pyinstaller==6.17.0
# Set the python shortcut you are using on your OS
PYTHON=python3.12
```

The version of the *"pyinstaller"* and the Python(shortcut) you are using.

Run the script.

```
$ bash create_package_linux.sh
```

Chapter 3

How to use it

Use the *".exe"* file or the source code to play with it.

Supported application formats

The current version supports the following application formats

- setuptools format
- pyproject.toml format
- wheel format

Note

If you are setting up the environment for the first time, or after your application has changed, you MUST be connected to the internet to download and install the required dependencies

First step

All examples below are demonstrate the use of the *".exe"* file of the ***"Starter"*** application.

1. Run the ***"Starter"*** application for the first time to create folders and files structure to run your application.
    1. Via UI
        1. Click on ***"app_starter"*** or ***"app_starter.exe"***
    2. Via cmd
        1. Windows
            ```
            .\\app_starter.exe
            ```
        2. Linux
            ```
            $ ./app_starter
            ```

Second step (first and second step at once)

The first step has completed successfully. You should see the following structure(e.g. on Linux):

```
$app_environment
$cd app_environemnt
$ls -l
$d….app
..……...app_starter_config.json
d..….app_env
…….context.json
```

The folder "app_environment" contains following folders and files:

- ***Folders:*** *"app", "app_environemnt"*
- ***Files:*** *"app_starter_config.json", "context.json"*

Now it's time to specify the basic information about your application, such as where to find it, the parameters to run the application with, and the main file to launch. The straightforward way is configure the environment via the *"app_starter_config.json"* file. After the first run of the ***"Starter"*** application, the configuration file contains:

```
{
  "app_files": {},
  "app_folder": "",
  "app_params": "",
  "main_file": ""
}
```

What to update

app_files = contains a list of files with theirs timestamps. This helps to detect changes in your application, is used internally

app_folder = here you can specify where you app is located, default value is set to *"app"* from environment_structure set during "First step" operation(structure is created). Two options here, first copy app code to folder "app_enviroment/app" or set it to *"path/to/my/app"*.

Here you can specify the location of your application's installation file or source code. By default *"app"* folder is used(this folder is created after *"First step"* is completed).

There are two options

- Copy your application files to the folder *"app_environment/app"*
- Update this value to point to a different location

app_params = the parameters to use when launching your application(in string format).

main_file = main file(with entry point), string format. The ***"Starter"*** app is searching for all files fitting the requirement for *"main file"* and start every one of them. This also means that during attempt to start unrelated *"main file"* can cause fail of starter. If you specify this option, it will try to run only specified one(if it can find it).

The main file(with the entry point), in string format. The ***"Starter"*** application searches for all the files that matches the criteria for a *"main file"* and attempts to start each of them. This means that trying to start unrelated *"main file"* can cause *"Starter"* to fail. By specifying this option, you can avoid this behavior and ensure that only the specified *"main file"* is launched .

Note

Escape your backslashes on Windows platform. Fill the *"main_file"* option.

On Windows platform, escape backslashes in file paths(the "main_file" option)

All at once

Another, more direct approach is to specify the application folder and main file via command line. However, the „step-by-step" is recommended.

- Windows

```
> .\\app_starter.exe –app_path \\path\\to\your\\app\\folder --main_file main_file.py
```

- Linux

```
$./app_starter –app_path /path/to/your/app/folder --main_file main_file.py
```

And repeat

Your environment for the application should be properly set up. This means that each time you launch the ***"Starter"*** *".exe"* file – either by clicking it or running it via command line – your application should start. The process may take a few seconds, as the ***"Starter"*** application checks whether your application has changed and reacts accordingly.

Note

The application is capable of processing and installing additional requirements outside of your application. This means that while your application may have all its required dependencies "locked in", there may be cases where you need to run code (e.g., using importlib) that has its own dependencies, which you might not know in advance. In such cases, the application provides a way to install these additional dependencies into the virtual environment.

Naming of extra requirement files

- *requirement*(e.g.)
  - dev_requirements.txt
  - requirements

Where to place

Your app's folder structure should look like(e.g. Linux):

```
$ ls -l <app>
$ pyproject.toml  tests  requirements.txt  src  README.md ...
```

How it works

Each time you start the ***"Starter",*** it must perform several steps before launching your application.

- Check if your application´s code has changed(depends on the format of the application you provided/supported)
  - Check if .py files or .whl files have changed
  - Using a simple "last update timestamp" to detect changes

- Find the main file to start your application
  - Searches all potential main file(this can take while, as it attempts to start each one until correct file is launched → blocking approach)
  - To speed up the process, specify the main file name in the configuration file

Your application changed

As mentioned, if changes are detected, the ***"Starter"*** application removes everything related to you app's environment and begins preparing the environment from scratch. Don't worry - your application files are preserved.

How it works

Retrieve the list of files with their timestamps, compare it to the previously stored list, and if any discrepancies are found, start over: recreate the environment, install dependencies, and install your application.

Chapter 4

Troubleshooting

Since this application is intended to be used as a *".exe"* app without a UI, there are only two ways to detect if something goes wrong.
The two methods are very similar; the only difference is how you access them.

Logging

The ***"Starter"*** application logs to a file named "*app_starter.log*", located next to the source code.
The log file is configured as a rotating log, with a maximum size of 1 MB and up to 3 backup files retained in history.

"Raw source code" format(e.g. Linux)

```
./
../
app_starter.log
create_package_win.bat*
HOWTO.pdf
poetry.lock
pyproject.toml
README.md
src/
tests/
```

".exe file" format(e.g. Linux)

```
./
../
app_environment/
app_starter*
app_starter.log
 _internal/
..few extra files
```

Command-line logging

Currently, the application uses a combination of subprocesses, which may cause some messages or errors to be missed (this is already on the TODO list).
As a result, you might not know what is happening—or not happening—for some time.
You can run the app's *".exe"* file from the command line to monitor progress.
Both the "***Starter"*** application and the application you are launching run in a single process, with no threads (at least for now).

Linux

- Open a normal terminal.
- Navigate to the location of Starter's .exe file and run it(follow any additional instructions).
- You should see the progress of what is happening, similar to the example below.

```
Installing pip ...
Collecting pip
  Using cached pip-25.3-py3-none-any.whl.metadata (4.7 kB)
Using cached pip-25.3-py3-none-any.whl (1.8 MB)
Installing collected packages: pip
Successfully installed pip-25.3
Installation of pip finished.
done.
Installing setuptools …
…….
……..
```

Windows

- Open the terminal/command line(type cmd in the search box), Do not use the powershell.
- Navigate to the location of Starter's .exe file and run it(follow any additional instructions).
- You should see progress of what is happening, similar to the example below.
    - See the Linux example above

Everything is broken

If your environment becomes compromised, simply remove the following:

- *dist/app_starter/app_environment*
- on Windows
    - *dist/app_starter/app_environment*
    - *dist/Lib*
    - *dist/Scripts*

And start over. If it still doesn’t work, check whether your applications works correctly in “stand-alone” mode or if ***“Starter”*** application can handle your application in the first place. Good luck!

Chapter 5

Development

Add dependencies as needed using **Poetry**.

Run tests

Use pytest and coverage to test your application.

```
$pytest ./tests
$coverage run -m pytest ./tests/
```

Code coverage

Use coverage to check the current level of code coverage.

```
$coverage run -m pytest ./tests/
$coverage report -m
```

Changes, new ideas, cooperation

Everyone reading this documentation is welcome to join and help develop and extend this application so we can get the most out of it. If you have an idea, fix, suggestion, or anything else, please don't hesitate to contact me. I'm looking forward to cooperation and fresh perspectives.

If you have already turned your idea into code, please create a pull request. I will review it as soon as possible and provide feedback.

Below, I have listed some points that need to be **DONE** and some that **CAN** be done.

Future

Technical debt

1) Add Makefile for Linux(e.g. run tests, create package, etc.)
2) Add more unit tests
3) Add automatic generation of the documentation
4) Add autotomatic generation of code coverage statistics
5) Support more types of Python applications
6) Improve logic to determine the type of Python application to start (e.g., required files, dependency installation, general file processing).
7) Add setuptools support for "offline" usage

New ideas

1) Dynamic loading of the application using the importlib module, without need to install the app via pip.